[Nom de l'établissement]
[Filière / département]

# Système de suivi des rendez-vous et d'estimation du risque de non-présentation

Rapport de stage – conception et réalisation d'un prototype

| | |
|---|---|
| **Étudiant(e)** | [Prénom NOM] |
| **Établissement** | [Nom de l'école / université] |
| **Organisme d'accueil** | [Nom de l'organisme] |
| **Encadrant(e) en entreprise** | [Nom et fonction] |
| **Encadrant(e) pédagogique** | [Nom et fonction] |
| **Période du stage** | [Date de début – date de fin] |
| **Année universitaire** | [AAAA–AAAA] |

[Ville], [mois et année de remise]

# Résumé

Ce rapport décrit un prototype logiciel de gestion et d'analyse de rendez-vous, centré sur l'estimation expérimentale du risque de non-présentation (*no-show*). Le dépôt étudié assemble une API REST écrite avec FastAPI, une base relationnelle PostgreSQL accessible par SQLAlchemy, une interface analytique Streamlit, un classifieur XGBoost chargé depuis un artefact Joblib et un assistant de suggestions utilisant un service Ollama local.

Le code comporte notamment des opérations de consultation et de mise à jour des patients, médecins et rendez-vous, une authentification par mot de passe haché et jeton JWT, un calcul de trois variables temporelles et un tableau de bord permettant de consulter des indicateurs. Il comprend également des tests unitaires et des contrôles de validation. Ces éléments constituent un état technique de prototype, et non une solution clinique validée ou une preuve de conformité réglementaire. Les données générées par le dépôt sont synthétiques ; aucun résultat de performance réel n'est fourni par le code examiné.

**Mots-clés :** rendez-vous, no-show, FastAPI, Streamlit, PostgreSQL, apprentissage automatique, protection des données.

# Note de lecture et hypothèses

Le rapport est établi à partir des fichiers présents dans le dépôt `healthcare-noshow-system` au moment de son examen. Il ne présume ni du contexte réel du stage ni d'un déploiement en établissement de santé. Les champs rouges de la page de garde sont à renseigner par l'étudiant. Les résultats mesurés, l'environnement d'exécution réellement utilisé, les choix d'architecture validés par les encadrants et les éventuelles évolutions postérieures au dépôt doivent être complétés à partir de preuves vérifiables.

Dans la suite, les éléments qualifiés d'« en place » sont repérés dans le code. Les éléments qualifiés d'« à réaliser / vérifier » sont des recommandations, des objectifs de feuille de route ou des obligations d'évaluation qui ne doivent pas être confondus avec une fonction déjà livrée. Aucun dossier patient réel n'a été consulté pour élaborer ce rapport.

# Table des matières

**Résumé** i

**Note de lecture et hypothèses** ii

**1 Introduction et contexte** 1

1.1 Problématique . . . 1

1.2 Objectifs du prototype . . . 1

1.3 Périmètre et état des preuves . . . 2

**2 Architecture et composants** 3

2.1 Vue d'ensemble . . . 3

2.2 Échanges et frontières de confiance . . . 3

2.3 Technologies et lancement . . . 4

**3 API, authentification et persistance** 5

3.1 API FastAPI . . . 5

3.2 Authentification et audit . . . 6

3.3 Schéma relationnel . . . 7

**4 Tableau de bord Streamlit** 8

4.1 Fonctions visibles . . . 8

4.2 Limites d'usage et d'échelle . . . 8

**5 Données et pipeline d'apprentissage automatique** 10

5.1 Variables et construction des caractéristiques . . . 10

5.2 Entraînement et inférence . . . 10

5.3 Résultats : champs à compléter, sans extrapolation . . . 11

**6 Assistant génératif local** 12

6.1 Implémentation observée . . . 12

6.2 Limites . . . 12

**7 Sécurité, confidentialité et limites réglementaires** 13

7.1 Contrôles présents . . . . . 13
7.2 Contrôles manquants ou à confirmer . . . . . 13
7.3 CNDP et loi marocaine 09-08 : prudence nécessaire . . . . . 13
7.4 Performances et équité : pas de validation clinique . . . . . 14

**8 Tests, qualité et reproductibilité 15**
8.1 Tests présents . . . . . 15
8.2 Ce que les tests ne prouvent pas . . . . . 15

**9 Feuille de route et recommandations 17**

**10 Conclusion 19**

**Références techniques 20**

**A Checklist de personnalisation avant remise 21**

# Chapitre 1

# Introduction et contexte

## 1.1 Problématique

Les rendez-vous manqués peuvent compliquer l'organisation d'un service de santé. Un outil de suivi peut aider le personnel à repérer les rendez-vous à examiner et à proposer des actions organisationnelles telles qu'un rappel ou une demande de confirmation. Une telle aide ne remplace ni l'évaluation par le personnel ni une décision clinique.

Le projet étudié réunit une gestion élémentaire de données de rendez-vous et une estimation de risque expérimentale. Son objectif technique observable est de relier une API de persistance et de prédiction à une interface de visualisation, puis d'exposer, sur demande, des suggestions générées par un modèle de langage exécuté localement.

## 1.2 Objectifs du prototype

— gérer les enregistrements des patients, des médecins et des rendez-vous ;

— contrôler les accès à l'API au moyen d'une authentification par jeton ;

— calculer et exposer une estimation de probabilité de no-show, seule ou par lot ;

— visualiser les rendez-vous d'une date, des statistiques descriptives et un historique patient dans Streamlit ;

— expérimenter des suggestions de suivi non médicales avec Ollama local ;

— expliciter les limites de sécurité, de confidentialité et de validation avant toute perspective de mise en service.

## 1.3 Périmètre et état des preuves

| **Sujet** | État constaté dans le dépôt |
|---|---|
| **API et données** | **En place** CRUD partiel des entités principales, validation Pydantic et couche SQLAlchemy. |
| **Authentification** | **En place** connexion par formulaire OAuth2, mot de passe bcrypt et JWT signé à expiration. |
| **Interface** | **En place** tableau de bord Streamlit connecté à l'API, consultation et graphiques. |
| **Modèle** | **En place** artefact pré-entraîné et script de réentraînement ; performance clinique **À réaliser / vérifier**. |
| **Assistant** | **En place** appel Ollama limité à une adresse locale et à des variables numériques ; validation opérationnelle et conformité **À réaliser / vérifier**. |
| **Réglementation** | Analyse préparatoire seulement ; aucune conformité CNDP ou à la loi marocaine 09-08 n'est revendiquée. |

# Chapitre 2

# Architecture et composants

## 2.1 Vue d'ensemble

Le dépôt est organisé en modules Python plutôt qu'en paquet applicatif complexe. `main.py` crée l'application FastAPI, charge `model.pkl`, déclare les routes et assemble leurs dépendances. `models.py` décrit les tables SQLAlchemy, `schemas.py` les contrats Pydantic et `database.py` la configuration de la connexion. Le tableau de bord (`dashboard.py`) consomme l'API via HTTP. Le script `train_model.py` et le module `feature_engineering.py` portent le pipeline de données et de modèle. `assistant_agent.py` encapsule la requête au service Ollama.

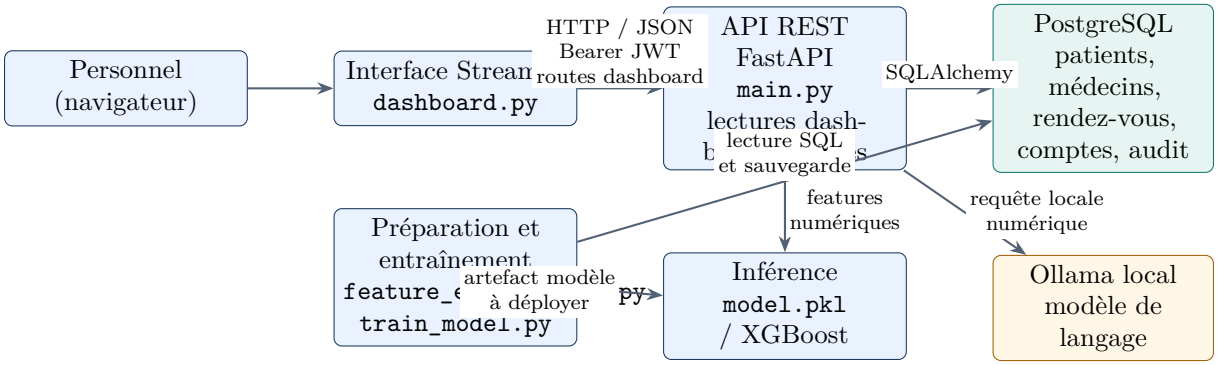



FIGURE 2.1 – Architecture logique observée dans le dépôt. La base et le service Ollama doivent être configurés séparément ; le schéma ne représente pas un déploiement de production validé.

## 2.2 Échanges et frontières de confiance

Le tableau de bord ne dialogue pas directement avec PostgreSQL : il envoie des requêtes à l'API et détient le jeton pendant la session Streamlit. L'API lit ou écrit les dossiers via SQLAlchemy, puis réalise l'inférence sur des variables numériques. Lorsqu'une suggestion est demandée, l'API relaie au processus Ollama uniquement le score, sa classe de risque, le délai de réservation, le jour de la semaine et l'historique no-show. Le nom, le téléphone, l'adresse électronique et le motif de visite ne sont pas inclus dans le message adressé au modèle de langage.

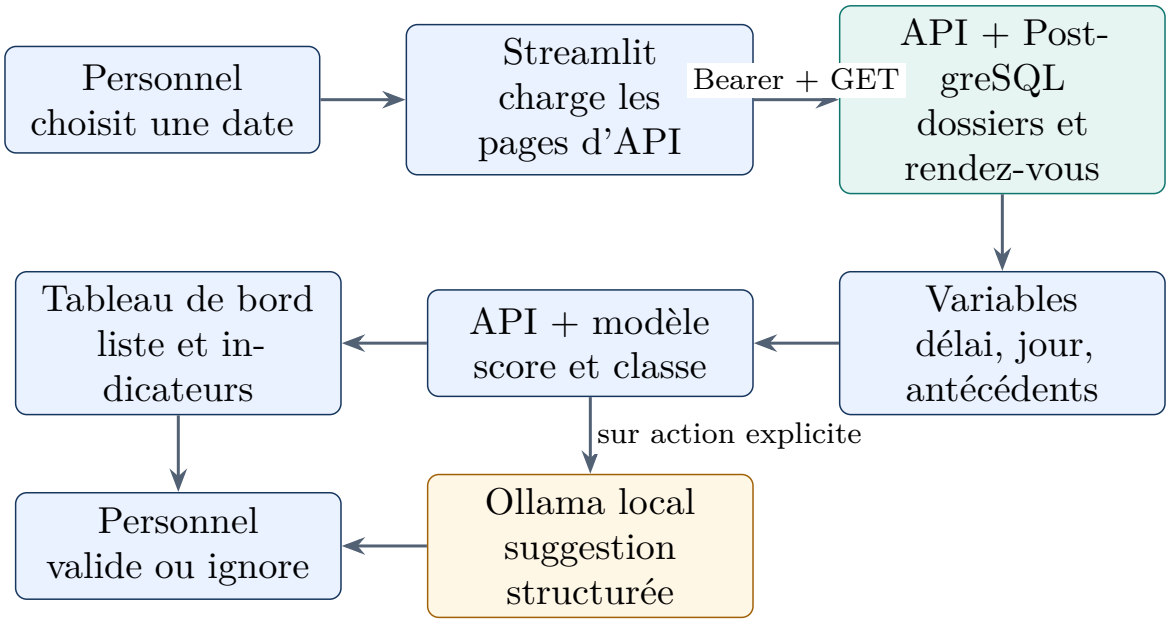



FIGURE 2.2 – Parcours d'une consultation et d'une suggestion. Le diagramme représente la logique codée, pas une décision automatisée : toute suggestion est destinée à une revue humaine.

## 2.3 Technologies et lancement

Les dépendances épinglées dans `requirements.txt` incluent FastAPI, Uvicorn, SQLAlchemy, Alembic, psycopg2, Pydantic, Streamlit, Plotly, pandas, scikit-learn, XGBoost, Joblib, bcrypt, PyJWT et requests. Le dépôt demande Python 3.10 ou plus récent et PostgreSQL. Les variables de configuration figurent comme exemples dans `.env.example` ; le fichier `.env` ne doit pas être publié.

Le README indique les commandes de démarrage suivantes après installation, configuration et application de la migration :

— `alembic upgrade head` pour créer les tables absentes ;

— `python create_user.py` pour créer un compte initial ;

— `uvicorn main:app -reload` pour démarrer l'API ;

— `streamlit run dashboard.py` pour démarrer le tableau de bord.

L'URL par défaut de l'API est `http://127.0.0.1:8000` ; la documentation OpenAPI est servie par FastAPI à `/docs`. Le mode `-reload` est un mode de développement, non une recommandation de déploiement.

# Chapitre 3

# API, authentification et persistance

## 3.1 API FastAPI

Les requêtes sont validées avec les schémas Pydantic. Les routes sauf `POST /auth/login` sont enregistrées sous un routeur protégé par `get_current_user`. Les listes acceptent une pagination `limit`/`offset` ; les listes patients et médecins ont une limite par défaut de 100 et maximale de 500. La liste des rendez-vous accepte aussi `from_date` et `to_date`. Une plage inversée est rejetée. Des routes spécifiques au tableau de bord fournissent des consultations filtrées, des agrégats calculés en base et une recherche patient bornée, afin d'éviter de charger systématiquement toutes les lignes dans Streamlit.

| Méthode | Route | Fonction et restrictions constatées |
|---|---|---|
| POST | `/auth/login` | Connexion par formulaire OAuth2 ; retourne un jeton bearer ou une réponse 401. Route publique nécessaire à l'obtention du jeton. |
| POST | `/predict` | Estimation pour un rendez-vous à partir de `delai_jours`, `jour_semaine` et `nb_no_show_precedents`. |
| POST | `/predict/batch` | Inférence par lot ; entre 1 et 500 éléments, un résultat par élément. |
| POST | `/assistant/no-show` | Génère une suggestion de suivi à partir du score, de sa classe et des trois variables numériques ; accès protégé. |
| POST / GET / GET par ID / PUT | `/patients` et `/patients/{patient_id}` | Création, liste paginée, lecture et mise à jour d'un patient. Aucune route DELETE n'est définie. |
| POST / GET / GET par ID / PUT | `/doctors` et `/doctors/{doctor_id}` | Création, liste paginée, lecture et mise à jour d'un médecin. Aucune route DELETE n'est définie. |
| POST / GET / GET par ID / PUT | `/appointments` et `/appointments/{appointment_id}` | Création, liste paginée et filtrée par date, lecture et mise à jour d'un rendez-vous. Aucune route DELETE n'est définie. |

| Méthode | Route | Fonction et restrictions constatées |
|---|---|---|
| GET | `/dashboard/appointments` | Requiert `appointment_date` ; joint le nom du patient et retourne les rendez-vous de cette date, avec délai, jour de semaine et nombre de no-shows aux dates antérieures. Pagination bornée à 500 éléments par réponse. |
| GET | `/dashboard/statistics` | Retourne la date la plus récente ainsi que des comptes par mois et par motif ; les agrégations sont exécutées par la base. |
| GET | `/dashboard/patients` | Recherche facultative par nom (`query`, 100 caractères au plus), pagination par défaut de 50 et limite maximale de 100 ; ne retourne que l'identifiant et le nom. |
| GET | `/dashboard/patients/{patient_id}/history` | Renvoie les comptes total/no-show du patient et une page d'historique limitée à 500 rendez-vous ; accepte `limit`/`offset` et retourne 404 si le patient n'existe pas. |

Les schémas imposent notamment des formats d'adresse électronique, des longueurs de champs, une date de naissance non future, une date de rendez-vous non antérieure à la réservation et un statut appartenant à une liste codée (`no_show`, `confirmé`, `confirmed`, `scheduled`, `cancelled`, `annulé`, `présent`, `attended`, `completed`). Les paramètres de prédiction sont bornés et le niveau de risque fourni à l'assistant doit correspondre au score suivant les seuils définis dans `risk.py`.

## 3.2 Authentification et audit

Le mot de passe est stocké sous forme de hachage bcrypt. Le script `create_user.py` valide un nom d'utilisateur, impose au moins huit caractères au mot de passe (et au plus 72 octets UTF-8) et accepte les rôles `admin` ou `staff`. L'API retourne un JWT signé en HS256, portant le sujet utilisateur et une date d'expiration (60 minutes par défaut, configurable par `ACCESS_TOKEN_EXPIRE_MINUTES`). Une clé `SECRET_KEY` d'au moins 32 caractères est requise au démarrage.

La dépendance d'authentification vérifie le jeton, retrouve l'utilisateur en base et écrit un enregistrement d'audit avec acteur, méthode HTTP, gabarit de route et horodatage. Le code n'enregistre pas le contenu de la requête dans cet audit. L'authentification n'implémente cependant pas de permissions distinctes par rôle : les rôles sont reconnus comme valides, mais `admin` et `staff` accèdent au même ensemble de routes. Les échecs de connexion ne sont pas audités ni limités en débit dans le code examiné.

## 3.3 Schéma relationnel

Le modèle SQLAlchemy contient cinq tables applicatives. Les types et contraintes ci-dessous sont ceux des modèles et de la migration initiale.

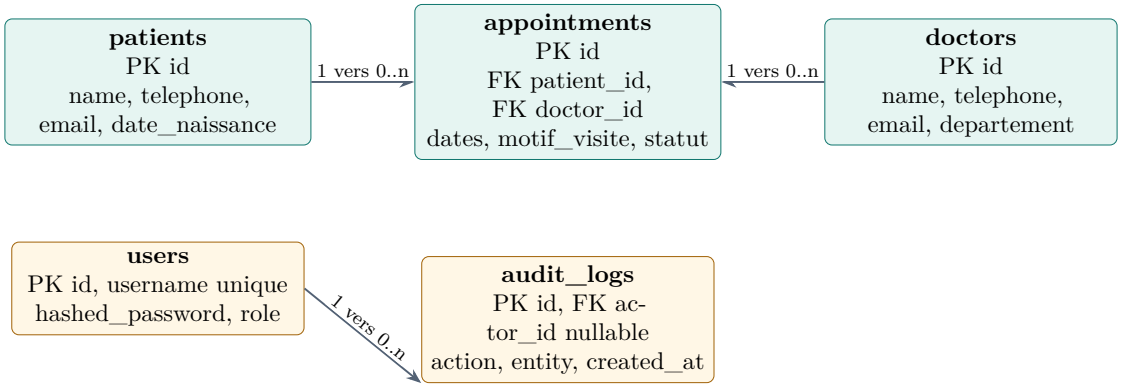



FIGURE 3.1 – Relations fonctionnelles définies par les clés étrangères. Les comptes et journaux ne sont pas reliés aux rendez-vous.

| Table | Attributs principaux | Rôle et remarques |
|---|---|---|
| `patients` | `id` PK, `name`, `telephone`, `email`, `date_naissance` | Informations d'identification et de contact ; les champs sont obligatoires. |
| `doctors` | `id` PK, `name`, `telephone`, `email`, `departement` | Informations d'identification et d'affectation du médecin. |
| `appointments` | `id` PK, `patient_id` FK, `doctor_id` FK, `date_reservation`, `date_rdv`, `motif_visite`, `statut` | Rendez-vous relié à un patient et un médecin. Index sur les deux clés étrangères et index composite `(date_rdv, id)` adapté au tri par date puis identifiant. |
| `users` | `id` PK, `username` unique, `hashed_password`, `role` | Comptes applicatifs ; contrainte SQL sur les rôles `admin` et `staff`. |
| `audit_logs` | `id` PK, `actor_id` FK nullable, `action`, `entity`, `created_at` | Trace les accès authentifiés ; index sur l'acteur et l'horodatage ; suppression du compte associé préserve le journal en mettant l'acteur à NULL. |

La configuration accepte soit une `DATABASE_URL`, soit les variables `DB_USER`, `DB_PASSWORD`, `DB_HOST`, `DB_PORT` et `DB_NAME`. Le constructeur SQLAlchemy compose les paramètres séparément pour éviter les problèmes d'échappement du mot de passe. Alembic applique une migration initiale qui crée les tables manquantes et vérifie les index. Elle ne modifie pas les colonnes d'une table déjà présente et est volontairement irréversible afin de protéger les données existantes ; les divergences de schéma héritées exigent donc une migration explicite. Une migration ultérieure en avant remplace l'index simple `ix_appointments_date_rdv` par l'index composite `(date_rdv, id)` lorsque l'index historique existe. Elle vérifie les colonnes existantes, préserve les rendez-vous et ne définit pas de rétrogradation ; l'application en production doit être planifiée et contrôlée comme toute évolution de schéma.

# Chapitre 4

# Tableau de bord Streamlit

## 4.1 Fonctions visibles

L'interface fournit un formulaire de connexion, une déconnexion, un filtre par date, une liste des rendez-vous de la journée enrichie par des scores, trois indicateurs (volume de rendez-vous, nombre de risques élevés, risque moyen), ainsi que deux vues statistiques : le taux mensuel de no-show et le taux par motif. Les listes et statistiques sont alimentées par les routes dédiées `/dashboard/appointments` et `/dashboard/statistics` : les rendez-vous sont filtrés dans la base avant affichage, les variables d'inférence sont préparées côté API et les agrégats sont calculés côté base. L'onglet de recherche patient appelle `/dashboard/patients` et fournit une recherche par nom limitée à 100 caractères avec au plus 100 résultats par page (50 par défaut). La consultation de l'historique est paginée via `/dashboard/patients/{patient_id}/history` ; les totaux de rendez-vous et de no-shows sont calculés côté base, tandis qu'au plus 500 entrées d'historique sont retournées par requête. Une action explicite ouvre l'assistant pour un rendez-vous sélectionné.

Les requêtes HTTP utilisent le jeton bearer conservé dans l'état de session. Le client a un délai d'attente de dix secondes par défaut et traite une réponse 401 en effaçant le jeton. L'appel GenAI utilise un délai de 90 secondes. Pour la date sélectionnée, le tableau de bord demande des pages d'au plus 500 rendez-vous déjà filtrés et enrichis par l'API, puis envoie les variables d'inférence en lots à `/predict/batch`. Les données patient utilisées pour l'onglet de recherche et d'historique sont récupérées à la demande et bornées par pagination, plutôt que de télécharger tout le répertoire.

## 4.2 Limites d'usage et d'échelle

Cette interface est un outil de consultation, pas une application complète d'administration : aucun formulaire de saisie ou de suppression n'est codé dans Streamlit, et les opérations de création ou de mise à jour sont uniquement disponibles comme routes d'API. Les principales lectures du tableau de bord sont désormais filtrées/agrégées côté base et l'historique est borné par pages ; cela réduit les transferts inutiles sans constituer une mesure de performance validée. La recherche d'un patient renvoie au plus une page de résultats à la fois, et l'interface actuelle ne propose pas de navigation de pages dans les résultats de recherche. Les plans de requête, index utiles, temps de réponse et comportement sous charge restent à mesurer sur un jeu de test représentatif.

Le tableau de bord manipule en mémoire des identifiants, noms et motifs de visite pour afficher ses vues, même si ces valeurs ne sont pas transmises à l'assistant GenAI. Il ne doit donc pas être

traité comme une interface déjà autorisée à recevoir de vraies données de santé.

# Chapitre 5

# Données et pipeline d'apprentissage automatique

## 5.1 Variables et construction des caractéristiques

Le module `feature_engineering.py` valide la présence des identifiants, dates et statuts, rejette les valeurs manquantes et les dates de rendez-vous antérieures à la réservation, puis trie les lignes par patient, date et identifiant. Les trois caractéristiques utilisées par le modèle sont :

— `delai_jours` : nombre de jours entre réservation et rendez-vous ;

— `jour_semaine` : jour de la semaine du rendez-vous (0 à 6) ;

— `nb_no_show_precedents` : nombre de no-shows du patient aux dates strictement antérieures.

Pour éviter qu'un rendez-vous n'influence les autres rendez-vous du même jour, les no-shows sont agrégés par patient et par date, puis le cumul est décalé d'une date avant d'être associé aux lignes du jour. La cible est binaire : statut `no_show` contre les statuts d'assiduité observée (`confirmé`, `confirmed`, `présent`, `attended`, `completed`). Les statuts d'annulation et les rendez-vous encore programmés ne servent pas de négatifs au réentraînement.

## 5.2 Entraînement et inférence

`train_model.py` lit dans PostgreSQL les colonnes d'identifiant patient, dates et statut, construit les variables, puis entraîne un `XGBClassifier` à 200 estimateurs avec une séparation chronologique approximativement 80/20 sur les dates distinctes. Le code vérifie que les partitions sont exploitables et que l'entraînement contient les deux classes. Il calcule accuracy, précision, rappel, F1, ROC-AUC lorsque cette métrique est définissable, PR-AUC et score de Brier, puis écrit le modèle dans `model.pkl`. L'inférence dans `main.py` retourne une classe prédite, une probabilité arrondie à quatre décimales et une bande de risque : faible si score inférieur à 0,3 ; moyen de 0,3 inclus à 0,6 exclu ; élevé à partir de 0,6.

Le fichier `model.pkl` présent dans le dépôt est un artefact pré-entraîné. Le code ne fournit pas, à lui seul, de provenance complète de cet artefact, de versionnement des jeux de données ou de rapport de validation clinique. Le script de génération produit des rendez-vous synthétiques de façon déterministe (graine par défaut 42) ; il insère sans supprimer les données existantes sauf option explicite `-replace`. Cette option tronque les rendez-vous et est destructive ; elle n'a pas été exécutée dans le cadre de ce rapport.

## 5.3 Résultats : champs à compléter, sans extrapolation

Aucun score mesuré n'est affirmé ici. Les métriques imprimées par `train_model.py` doivent être calculées sur un jeu de données et un protocole documentés, puis reportées avec leurs limites. L'évaluation sur données synthétiques ne démontrerait pas les performances sur une population réelle.

| **Élément du compte rendu** | Valeur vérifiée à compléter par l'étudiant |
|---|---|
| **Origine, période et effectif du jeu évalué** | **[À documenter ; préciser données synthétiques ou autorisées]** |
| **Date de séparation et nombres train/test** | **[Valeurs issues d'une exécution reproductible]** |
| **Accuracy, précision, rappel et F1** | **[Résultats mesurés avec définition du seuil]** |
| **ROC-AUC, PR-AUC et score de Brier** | **[Résultats mesurés ; préciser incertitudes et déséquilibre]** |
| **Évaluation par sous-groupes et impact opérationnel** | **[Non disponible dans le dépôt ; à mesurer avant tout usage]** |

# Chapitre 6

# Assistant génératif local

## 6.1 Implémentation observée

Le module `assistant_agent.py` construit un appel au point `/api/chat` d'Ollama. Par défaut, le modèle est `qwen2.5:3b` et le service est configuré sur `http://127.0.0.1:11434`. Le code refuse une URL qui n'utilise pas HTTP sur une adresse de boucle locale, ou qui contient des informations d'identification, une query ou un fragment. Il désactive les redirections et fixe un délai maximal de 90 secondes.

Le prompt demande une réponse en français au format JSON avec un résumé et une à trois actions choisies parmi quatre codes : rappel neutre, demande de confirmation, appel du personnel ou absence d'action supplémentaire. La réponse est validée par Pydantic ; les codes autorisés sont transformés en libellés compréhensibles et accompagnés d'un avertissement. En cas d'indisponibilité du service, de réponse HTTP non valide ou de contenu mal formé, l'API renvoie une erreur de service appropriée.

## 6.2 Limites

« Local » décrit ici la destination configurée de l'appel, non une garantie de sécurité, de confidentialité ou de conformité : le contrôle de boucle locale ne couvre pas à lui seul la machine hôte, l'accès local au service, le téléchargement du modèle, la journalisation, la rétention des prompts ou les composants système. L'interface et l'API doivent rester sous contrôle d'un opérateur, et toute suggestion doit faire l'objet d'une validation humaine. L'assistant est conçu pour des suggestions d'organisation, pas pour un diagnostic, une décision de soin ou une mesure punitive. Les tests simulent l'appel HTTP ; ils ne constituent pas une validation métier des productions d'un modèle Ollama réel.

# Chapitre 7

# Sécurité, confidentialité et limites réglementaires

## 7.1 Contrôles présents

Le dépôt contient des mesures utiles au stade du prototype : hachage bcrypt, jetons signés et expirants, validation des données d'entrée, pagination bornée, journalisation des requêtes authentifiées sans corps de requête, restrictions locales pour Ollama et exclusion de `.env` de Git. Le générateur de données ne supprime pas les rendez-vous par défaut.

## 7.2 Contrôles manquants ou à confirmer

Les éléments suivants ne sont pas attestés comme configurés ou éprouvés par le seul code :

— HTTPS de bout en bout, chiffrement et contrôle d'accès de la base, sauvegardes chiffrées et restauration testée ;

— limitation des tentatives de connexion, MFA, révocation de session, cycle de vie des comptes et permissions différentes selon les rôles ;

— politique de conservation et suppression des données et des journaux, supervision, alertes, gestion d'incident et audit d'intégrité ;

— information des personnes, gestion de leurs droits, minimisation validée et procédures d'archivage ou de correction ;

— analyse formelle des risques, inventaire des sous-traitants, de l'hébergement, des accès de support et d'éventuels transferts ;

— tests de sécurité et validation de configuration pour un environnement de production.

## 7.3 CNDP et loi marocaine 09-08 : prudence nécessaire

Le dépôt comporte un document préparatoire `SECURITY_PRIVACY.md` qui rappelle que ce prototype ne constitue pas une déclaration de conformité. Les informations liées à la santé nécessitent une analyse juridique et de gouvernance spécifique. Le responsable de traitement, les finalités, la base juridique, les formalités applicables auprès de la CNDP, l'information des personnes, leurs droits, la durée de conservation et les transferts éventuels doivent être déterminés par les

personnes compétentes et confirmés à partir des règles en vigueur. Ce rapport n'apporte pas de conseil juridique et ne conclut pas que le système satisfait à la loi 09-08 ou à une exigence de la CNDP.

Avant toute utilisation de données réelles, il faut obtenir les validations institutionnelles et juridiques requises, réaliser une analyse de risques, mettre en œuvre des protections techniques et organisationnelles adaptées et documenter les responsabilités. L'exécution locale d'un modèle ne dispense pas de ces vérifications. Aucune donnée patient réelle n'a été consultée pour préparer ce rapport.

## 7.4 Performances et équité : pas de validation clinique

Les scores sont produits par un modèle expérimental et ne doivent pas être utilisés seuls pour refuser, retarder ou prioriser l'accès aux soins. Le dépôt ne fournit aucune preuve d'évaluation prospective, de calibration sur données représentatives, de mesure des disparités entre sous-groupes, d'évaluation d'impact ou de validation clinique. Les classes de risque sont des seuils techniques configurés, non des niveaux approuvés par une autorité médicale. La vérification des étiquettes, des rendez-vous dupliqués, des annulations et des données manquantes reste nécessaire avant de tirer des conclusions.

# Chapitre 8

# Tests, qualité et reproductibilité

## 8.1 Tests présents

Le fichier `tests/test_project.py` couvre notamment :

— l'ingénierie des variables, l'exclusion des événements du même jour, l'ordre des colonnes et le rejet de données incohérentes ;

— les bornes des entrées Pydantic, les statuts et dates de rendez-vous, les seuils de classe de risque et la validation d'adresse électronique ;

— le découpage chronologique et le calcul des métriques d'entraînement, ainsi que l'exclusion des rendez-vous annulés ;

— les données envoyées à Ollama, le refus d'une URL distante et la validation de la réponse générée ;

— la création non destructive des tables manquantes par migration et des cas d'API : authentification, prédiction, lots et audit.

Les tests complémentaires `tests/test_dashboard_api.py` vérifient les routes de lecture du tableau de bord, leurs limites et les réponses paginées ; `tests/test_query_indexes.py` vérifie l'index composite et le remplacement de l'index historique par migration en avant sans perte des lignes existantes. Les tests utilisent une base SQLite en mémoire ou de test, des données fabriquées et des appels externes simulés. Le workflow GitHub Actions `.github/workflows/tests.yml` installe les dépendances, compile les modules Python, exécute la suite unittest et vérifie la découverte de l'historique Alembic sur les événements push et pull request.

**Résultat vérifié le 9 novembre 2026 :** la commande ci-dessous a exécuté 27 tests avec succès dans l'environnement de développement. Il s'agit de tests unitaires utilisant notamment SQLite et des services externes simulés ; ce résultat ne démontre ni les performances en production, ni l'interopérabilité complète avec PostgreSQL, ni une validation clinique :

**Commande :** `python -m unittest discover -s tests -v`
**Résultat observé :** `Ran 27 tests; OK`

## 8.2 Ce que les tests ne prouvent pas

Les tests unitaires ne démontrent pas le fonctionnement sur PostgreSQL configuré pour un déploiement, la robustesse sous charge, la sécurité d'une configuration réseau, la qualité des prédictions sur des données de terrain ou la conformité réglementaire. Ils ne remplacent pas

des tests d'intégration et de bout en bout, des tests d'autorisation par rôle, une analyse de dépendances ou des essais utilisateur.

# Chapitre 9

# Feuille de route et recommandations

Le fichier `IMPROVEMENT_ROADMAP.md` organise les évolutions en priorités. Le tableau suivant distingue les recommandations de l'état implémenté.

| Priorité | Travaux recommandés | Résultat attendu |
|---|---|---|
| P0 | **À réaliser / vérifier** Revoir l'autorisation de bout en bout, les rôles, la loi 09-08/CNDP, le responsable et les finalités ; exiger HTTPS, gestion sûre des secrets, chiffrement des connexions et sauvegardes ; ajouter limitation des tentatives, MFA, gestion des comptes, rétention d'audit et restauration testée. | Réduire les risques de lancement et établir les conditions de traitement légitime et sécurisé avant toute donnée réelle. |
| P1 | **En place** Les lectures dédiées du tableau de bord filtrent les rendez-vous, calculent les statistiques et bornent recherche/historique côté API et base ; une migration en avant ajoute l'index `(date_rdv, id)`. **À réaliser / vérifier** Mesurer les plans de requête, index et temps de réponse sur un jeu représentatif, puis ajouter santé et disponibilité, erreurs structurées, suivi, traçage, CI élargie et environnements reproductibles. | Confirmer les gains réels et améliorer la fiabilité, la performance et l'exploitabilité du service. |

| Priorité | Travaux recommandés | Résultat attendu |
|---|---|---|
| P2 | **À réaliser / vérifier** Versionner les jeux, variables, modèles et évaluations ; documenter les étiquettes ; évaluer chronologiquement et par groupes, calibration, impact opérationnel et dérive ; prévoir une revue humaine et une correction des données. | Établir si le modèle apporte une aide mesurable, équitable et acceptable, sans automatiser une décision de soin. |
| P3 | **À réaliser / vérifier** Ajouter suppression/archivage avec droits et audit, recherche et filtres plus complets, accessibilité et localisation, versionnage API, exemples OpenAPI, journalisation prudente, tests d'intégration et de charge sur données synthétiques. | Faire évoluer le prototype vers une application maintenable et utilisable. |

# Chapitre 10

# Conclusion

Le dépôt met en œuvre une chaîne logicielle cohérente pour expérimenter une application de rendez-vous : API FastAPI, stockage relationnel, authentification JWT, tableau de bord Streamlit, calcul de caractéristiques temporelles, inférence XGBoost et suggestions de workflow via Ollama local. Les migrations, les validations de schéma et des tests unitaires donnent une base de développement identifiable.

Cette cohérence technique ne suffit pas à autoriser un usage opérationnel dans un environnement de soins. Les performances n'ont pas été établies sur des données réelles représentatives, le système ne comporte pas toutes les protections ni procédures nécessaires, et aucune conformité à la CNDP ou à la loi 09-08 n'est déclarée. La prochaine étape est donc une phase de cadrage, d'évaluation et de conformité, avant tout traitement de données personnelles réelles ou toute utilisation de scores pour orienter une action.

# Références techniques

Les références principales utilisées pour décrire l'implémentation sont les fichiers du dépôt :

— `README.md` : installation, lancement, utilisation du modèle et de l'assistant ;

— `main.py`, `auth.py`, `schemas.py` : routes, sécurité applicative et validation ;

— `models.py`, `database.py`, `migrations/versions/` : schéma relationnel, connexion et migration ;

— `dashboard.py` : parcours de consultation et visualisations ;

— `feature_engineering.py`, `train_model.py`, `risk.py`, `model.pkl` : variables, entraînement, seuils et artefact d'inférence ;

— `assistant_agent.py` : intégration Ollama ;

— `tests/test_project.py` et `.github/workflows/tests.yml` : tests et automatisation ;

— `SECURITY_PRIVACY.md` et `IMPROVEMENT_ROADMAP.md` : limites et travaux à prévoir.

Pour les obligations applicables, consulter les ressources officielles de la CNDP (https://www.cndp.ma/) et solliciter les compétences juridiques et institutionnelles pertinentes. La liste ci-dessus n'est pas une validation juridique.

# Annexe A

# Checklist de personnalisation avant remise

1. Remplacer tous les champs **[entre crochets]** de la page de garde par des informations exactes ; ne pas inventer le nom de l'établissement, les personnes ou les dates.
2. Ajouter, si disponible, le contexte du stage réellement autorisé et les décisions confirmées par les encadrants, sans exposer de données personnelles ou de secret.
3. Compléter les mesures uniquement après exécution et archiver le protocole, la source des données et le résultat reproductible.
4. Maintenir la distinction entre fonctions présentes, évolutions planifiées et validations réglementaires encore requises.
5. Relire le rapport avec l'encadrant et vérifier la politique de confidentialité et de diffusion de l'établissement d'accueil.